

KritiShield AI

AI-Driven Cyber Resilience Platform for Critical Infrastructure

ET Hackathon — Project Documentation | July 2026

1. Problem Statement

Critical national infrastructure — hospitals, exam boards, government systems — is a growing target for cyberattacks. Incidents like the AIIMS Delhi ransomware attack and the CBSE data breaches show that most breaches are discovered weeks or months after the fact, because traditional tools rely on known malware signatures rather than behaviour.

- Over 70% of government IT infrastructure runs on end-of-life systems.
- Advanced Persistent Threats (APTs) move slowly and quietly to avoid signature-based detection.
- What's needed: a behavioural intelligence layer that flags anomalies, maps them to known attack patterns, and responds automatically — cutting detection time from weeks to hours.

2. What We Built

KritiShield AI is a working prototype of an autonomous cyber resilience platform. It simulates a live infrastructure environment, detects behavioural anomalies in real time, uses a multi-agent AI system to identify the attack pattern and explain it in plain English, and visualises everything on a security command-center dashboard.

3. Key Features

Backend & Infrastructure Simulation

- PostgreSQL-backed database tracking assets, live activity logs, incidents, and known vulnerabilities.
- Network activity simulation engine — generates continuous normal traffic and can inject realistic attack sequences (e.g. late-night credential misuse, cross-zone lateral movement) on demand.
- Topology engine (NetworkX) that computes asset relationships and outputs a node-and-edge graph for the UI.
- FastAPI service layer exposing asset metrics, alerts, and simulation triggers, plus endpoints to programmatically isolate assets or revoke credentials (automated containment).

AI & Intelligence Layer

- Vector store (RAG) seeded with public vulnerability data for semantic security-context retrieval.
- Behavioural anomaly engine that scores incoming logs and cross-zone traffic, producing a live deviation score D_s between 0.0 and 1.0.
- Multi-agent workflow that automatically fires when $D_s > 0.7$:
 - **Agent 1 – Threat Mapping** — maps the event sequence to known attack tactics and predicts the adversary's next move
 - **Agent 2 – Explainable AI Narrative** — generates a plain-English explanation of why a defensive action was triggered
- Structured output enforcement so agent responses map cleanly into the incident database.

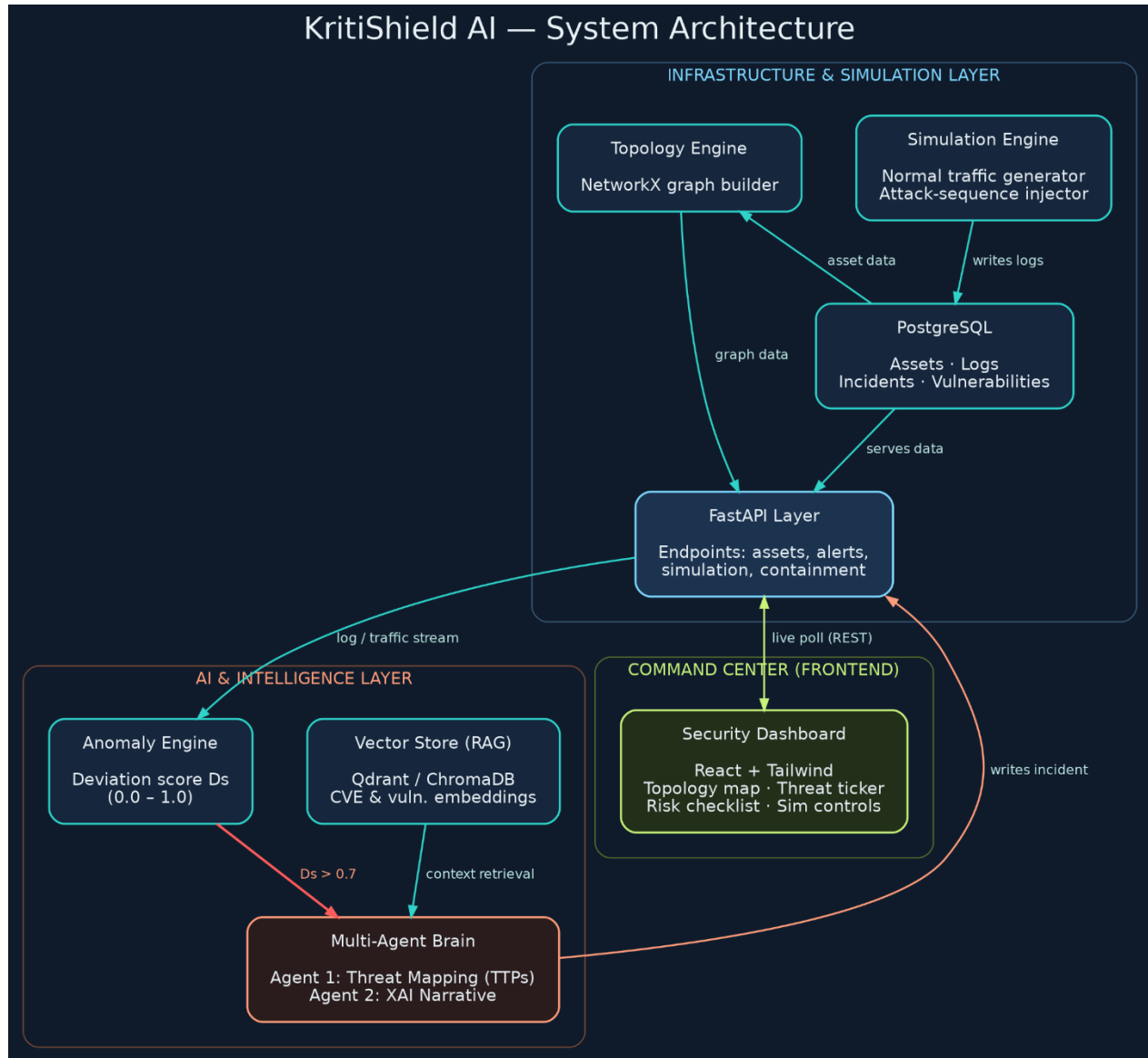
Frontend — Security Command Center

- Dark-themed dashboard built with React and Tailwind CSS.
- Interactive network topology map rendering live node/edge data from the backend.
- Lateral-movement indicators — connections flash red when an active traversal threat is flagged.
- Live threat ticker, network health metrics, and a prioritised vulnerability checklist.
- Simulation control panel letting judges pick a scenario and watch the attack unfold in real time.

- Real-time polling to keep graphs, metrics, and logs in sync with the backend.

4. System Architecture

The platform is split into three cooperating layers: a simulation and data layer, an AI reasoning layer, and a live dashboard. Simulated logs flow into the anomaly engine; once the deviation score crosses the threshold, the multi-agent system writes an explained incident back to the database, which the dashboard reflects in real time.



5. Data Flow (Step by Step)

- Simulation engine writes normal or attack-injected logs to PostgreSQL.
- Anomaly engine reads log/traffic streams and computes a deviation score D_s .
- If $D_s > 0.7$, the multi-agent workflow triggers: Agent 1 identifies the likely attack technique, Agent 2 writes a plain-English explanation.
- The finding is written back as a structured incident in the database via the FastAPI layer.
- The dashboard polls the API and updates the topology map, threat ticker, and metrics live.

6. Tech Stack

Layer	Technology
Backend framework	Python 3 + FastAPI
Database	PostgreSQL
Graph computation	NetworkX
Agentic framework	CrewAI / LangChain
Vector store (RAG)	Qdrant / ChromaDB
LLM inference	Llama 3 via fast-hosting API (e.g. Groq)
Frontend	React (ES6+), HTML5, CSS3
Styling	Tailwind CSS
Graph visualisation	Vis.js Network / Cytoscape.js

7. Deliverables

- Working prototype (backend + AI engine + dashboard)
- Architecture diagram
- Demo video